

Code Insight - An Educational Code Assessment Platform Using Fine-Tuned and Generative AI Models

Mr. Pratik Anil Patil
Department of Computer Engineering
KCT's Late G. N. Sapkal College of Engineering
Nashik, India
patilpratikani252004@gmail.com

Mr. Tanish Ashok Tambe
Department of Computer Engineering
KCT's Late G. N. Sapkal College of Engineering
Nashik, India
tanishtambe13@gmail.com

Mr. Mandar Prithviraj Deshmukh
Department of Computer Engineering
KCT's Late G. N. Sapkal College of Engineering
Nashik, India
deshabhi2004@gmail.com

Mr. Uday Kailas Jadhav
Department of Computer Engineering
KCT's Late G. N. Sapkal College of Engineering
Nashik, India
udayjadhav113@gmail.com

Abstract — The increasing demand for effective programming education highlights the need for intelligent systems that go beyond basic code evaluation. Traditional assessment platforms primarily focus on correctness and execution, often lacking meaningful feedback on code quality, efficiency, and conceptual understanding. This paper presents an educational code assessment platform that utilizes a hybrid artificial intelligence approach by combining fine-tuned and generative AI models. The proposed system integrates a fine-tuned CodeLlama 7B model for analyzing time and space complexity along with code structure, while a generative AI model (Gemini API) provides detailed explanations, optimization suggestions, and learning support. This combination enables both accurate technical evaluation and interactive educational feedback. Additionally, the platform supports automated execution, submission tracking, and a performance-based leaderboard to enhance student engagement. The hybrid approach improves the quality and interpretability of feedback compared to traditional methods, helping learners better understand programming concepts and improve code efficiency. The system demonstrates the potential of combining analytical and generative AI techniques to enhance modern programming education.

Keywords — AI Assisted Code Evaluation, Computational Complexity Estimation, Continuous Coding Practice Platform, Pretrained LLMs

I. INTRODUCTION

The rapid expansion of the software industry and digital technologies has significantly increased the demand for proficient programming skills. As a result, programming education has become a fundamental component of academic curricula and professional training programs. However, evaluating programming assignments effectively remains a challenging task, particularly in large classrooms and online learning environments. Traditional code assessment methods and existing automated platforms primarily focus on correctness and successful execution of programs, often overlooking important aspects such as code efficiency, structure, and conceptual clarity. This limitation restricts students from receiving comprehensive feedback necessary for improving their problem-solving abilities.

In recent years, artificial intelligence has emerged as a promising solution for enhancing automated code evaluation

systems. Machine learning and deep learning models have been applied to analyze code patterns, detect errors, and estimate computational complexity. At the same time, generative AI models have demonstrated the ability to produce human-like explanations and assist in understanding complex programming concepts. Despite these advancements, most existing systems rely on either analytical models or generative models independently, which leads to a gap between precise technical evaluation and meaningful educational feedback.

To address this challenge, this paper proposes an educational code assessment platform that employs a hybrid artificial intelligence approach. The system combines a fine-tuned CodeLlama 7B model for analyzing time complexity, space complexity, and code structure with a generative AI model (Gemini API) that provides detailed explanations, optimization suggestions, and conceptual guidance. By integrating these two approaches, the platform aims to deliver both accurate evaluation and interactive learning support within a single system.

In addition to AI-based analysis, the platform incorporates features such as automated code execution, submission tracking, and a performance-based leaderboard, making it suitable for academic institutions and self-learning environments. The primary objective of this work is to bridge the gap between automated assessment and pedagogy by providing students with actionable insights that enhance their coding skills and conceptual understanding.

II. LITERATURE SURVEY

The evaluation of programming assignments has evolved from manual assessment to automated systems aimed at improving scalability and consistency. Early automated code assessment tools primarily focused on compiling and executing programs against predefined test cases to verify correctness. While such systems are efficient in handling large numbers of submissions, they provide limited feedback, often failing to address deeper aspects such as algorithmic efficiency, code quality, and conceptual understanding. This limitation has motivated researchers to explore more intelligent approaches for code analysis.

Subsequent advancements introduced static analysis techniques and rule-based systems to evaluate code structure and detect common programming errors. These methods analyze syntax, control flow, and predefined coding standards to generate feedback. Although they improve the quality of evaluation compared to basic execution-based systems, their effectiveness is constrained by rigid rules and limited adaptability to diverse coding styles and problem-solving approaches.

With the emergence of machine learning, researchers began applying data-driven models to understand programming patterns and predict code quality. Supervised learning techniques have been used to classify code submissions, detect bugs, and estimate performance metrics such as time and space complexity. These approaches offer improved flexibility over rule-based systems; however, they often require large labeled datasets and may lack interpretability, making it difficult for learners to understand the reasoning behind the evaluation.

More recently, the development of large language models has significantly influenced the domain of code analysis and education. Generative AI models are capable of understanding code context and producing human-like explanations, suggestions, and even optimized code snippets. These models enhance the learning experience by providing descriptive feedback and conceptual clarity. Nevertheless, generative models alone may not always guarantee precise analytical accuracy, particularly in tasks such as complexity estimation and structured code evaluation.

To overcome the individual limitations of analytical and generative approaches, recent studies have explored hybrid AI systems that combine multiple models to leverage their complementary strengths. Such systems aim to integrate the precision of fine-tuned models with the interpretability of generative models. However, the application of hybrid AI specifically in educational code assessment platforms remains relatively underexplored, especially in terms of delivering both accurate technical evaluation and meaningful pedagogical feedback within a unified system.

Based on the identified gaps, this paper proposes a hybrid AI-based educational code assessment platform that combines fine-tuned and generative models to provide comprehensive evaluation and enhanced learning support. The proposed approach seeks to contribute to the advancement of intelligent educational tools by bridging the gap between automated assessment and effective pedagogy.

III. PROPOSED SYSTEM

This section describes the design and working of the proposed educational code assessment platform, which utilizes a hybrid artificial intelligence approach to provide both accurate evaluation and meaningful learning support. The methodology integrates automated code execution with AI-driven analysis to assess not only the correctness of a program but also its efficiency, structure, and conceptual quality.

3.1 System Overview

The proposed system follows a multi-stage pipeline where user-submitted code is processed through execution, analysis, and feedback generation modules. Initially, the user submits code through the platform interface. The code is then compiled and executed in a controlled environment to verify correctness using predefined test cases. Once execution is complete, the code is forwarded to the hybrid AI module for deeper analysis.

The hybrid AI module is the core component of the system, designed to combine the strengths of fine-tuned and generative models. This integration enables the system to deliver both precise technical insights and human-readable explanations, thereby supporting educational objectives.

3.2 Hybrid AI Framework

The proposed methodology employs two complementary AI components:

- 1. Fine-Tuned Analytical Model:**
A fine-tuned CodeLlama 7B model is used to analyze the structural and computational aspects of the submitted code. This model is trained to estimate time complexity, space complexity, and identify key code patterns. It provides structured outputs that reflect the efficiency and organization of the program. The use of a fine-tuned model ensures higher accuracy and consistency in analytical evaluation.
- 2. Generative AI Model:**
A generative AI model, accessed through the Gemini API, is used to generate detailed explanations, optimization suggestions, and conceptual guidance. This component interprets the code in a human-like manner and produces feedback that is easy for students to understand. It focuses on improving code readability, logic clarity, and learning outcomes.

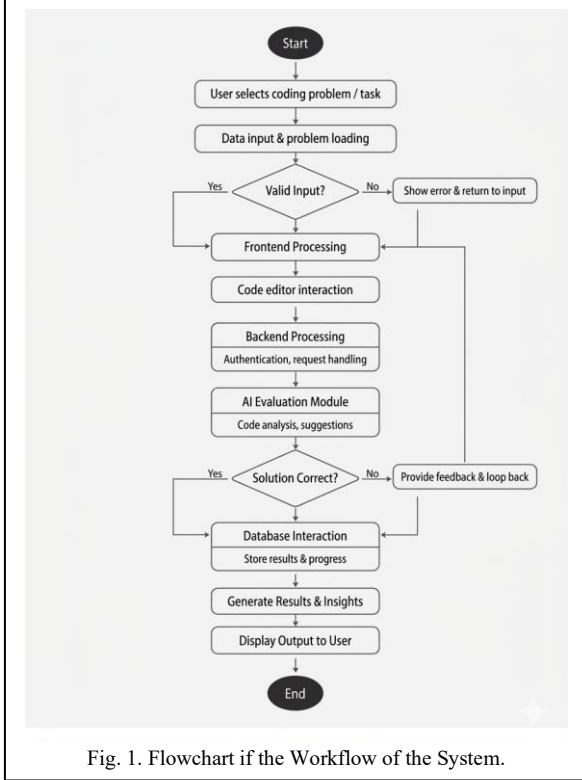
The outputs from both models are combined to produce a comprehensive evaluation report. The analytical model ensures technical correctness and precision, while the generative model enhances interpretability and educational value.

3.3 Workflow of the System

The overall workflow of the proposed system can be summarized as follows:

- 1. Code Submission:**
The user submits a program through the platform interface.
- 2. Compilation and Execution:**
The system compiles and executes the code against predefined test cases to verify correctness.
- 3. Preprocessing:**
The code is cleaned and formatted to ensure compatibility with AI models.
- 4. AI-Based Analysis:**
 - The fine-tuned model evaluates time complexity, space complexity, and code structure.
 - The generative model generates explanations and optimization suggestions.

5. **Result Aggregation:**
Outputs from both models are combined into a unified feedback report.
6. **Feedback Delivery:**
The system presents execution results, complexity analysis, and AI-generated suggestions to the user.



3.4 Educational Features Integration

In addition to AI-based analysis, the platform incorporates several features aimed at enhancing the learning experience. These include automated tracking of submissions, a performance-based leaderboard, and continuous feedback mechanisms. Such features encourage practice, competition, and self-improvement among students.

3.5 Key Advantages of the Proposed Methodology

The proposed hybrid approach offers several advantages over traditional systems:

- Provides both **accurate technical evaluation** and **interpretable feedback**
- Enhances **conceptual understanding** through detailed explanations
- Supports **scalable and automated assessment** for large user bases
- Bridges the gap between **evaluation and learning**

IV. SYSTEM ARCHITECTURE

The system architecture of the proposed platform is designed to ensure efficient processing of user submissions while integrating hybrid AI capabilities for comprehensive code evaluation and feedback. The architecture follows a modular

and layered design, allowing scalability, maintainability, and seamless interaction between different components.

4.1 Architectural Overview :

The platform is structured into four primary layers: the **User Interface Layer**, **Application Layer**, **Execution Layer**, and **AI Analysis Layer**, supported by a centralized **Database Layer**. Each layer performs a specific function in the overall workflow, enabling smooth data flow from code submission to feedback generation.

4.2 User Interface Layer :

The User Interface (UI) layer serves as the interaction point between users and the system. It is designed to provide an intuitive and responsive environment where users can write, edit, and submit code. The interface supports features such as problem statements, code editor integration, submission history, and leaderboard visualization. This layer communicates with the backend through API requests, ensuring real-time interaction and feedback display.

4.3 Application Layer :

The Application Layer acts as the central controller of the system. It handles user requests, manages session data, and coordinates communication between different modules. When a user submits code, this layer validates the input, forwards it for execution, and later routes it to the AI analysis components. It also aggregates results from various modules and formats them into a structured response for the user interface.

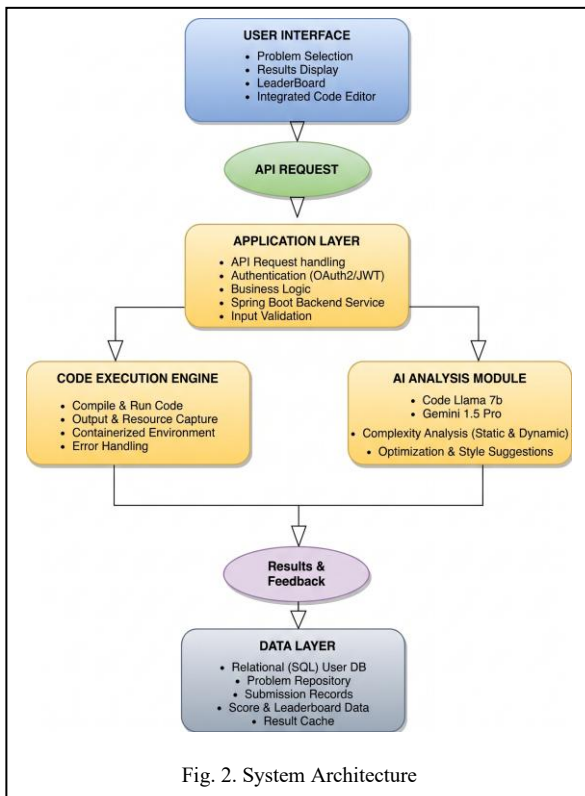
4.4 Execution Layer :

The Execution Layer is responsible for compiling and running the submitted code in a secure and isolated environment. It uses predefined test cases to verify the correctness of the program. This layer ensures that code execution is performed safely, preventing unauthorized system access or malicious operations. The output generated from this stage includes execution results, runtime behavior, and error messages, which are essential for initial evaluation.

4.5 AI Analysis Layer :

The AI Analysis Layer is the core of the proposed architecture, implementing the hybrid AI approach. It consists of two main components:

- **Analytical Model Component:**
This component uses a fine-tuned CodeLlama 7B model to analyze the submitted code. It evaluates time complexity, space complexity, and structural characteristics, producing structured outputs for precise technical assessment.
- **Generative Model Component:**
This component utilizes a generative AI model (Gemini API) to generate detailed explanations, optimization suggestions, and conceptual insights. It transforms technical analysis into human-readable feedback, enhancing the learning experience. The outputs from both components are combined within this layer to create a comprehensive evaluation report.



4.6 Database Layer :

The Database Layer stores all essential data, including user information, problem statements, code submissions, execution results, and analysis reports. It also maintains leaderboard data and historical performance records. This persistent storage enables tracking of user progress and supports analytics for system improvement.

4.7 Data Flow :

The overall data flow in the system follows a sequential process. The user submits code through the interface, which is processed by the application layer and sent to the execution layer for validation. After execution, the code and results are forwarded to the AI analysis layer for detailed evaluation. Finally, the aggregated feedback is stored in the database and displayed to the user through the interface.

4.8 Architectural Benefits :

The modular design of the system architecture offers several advantages:

- **Scalability:** Independent components allow easy expansion of system capabilities
- **Security:** Isolated execution environment ensures safe code processing
- **Flexibility:** Hybrid AI components can be updated or replaced without affecting other modules
- **Efficiency:** Parallel processing of execution and analysis improves response time

V. RESULTS AND DISCUSSIONS

This section presents the outcomes of the proposed educational code assessment platform and analyzes its effectiveness in terms of performance, accuracy, and educational impact. The evaluation focuses on the system's ability to provide meaningful feedback, accurately assess code efficiency, and enhance the overall learning experience.

5.1 Experimental Setup :

The system was tested using a diverse set of programming problems covering fundamental concepts such as loops, recursion, data structures, and algorithmic challenges. Multiple code submissions with varying levels of complexity and efficiency were provided as input to evaluate the robustness of the platform. The evaluation criteria included correctness of execution, accuracy of complexity estimation, quality of feedback, and system responsiveness.

5.2 Performance of Code Execution Module :

The execution module successfully compiled and ran user-submitted programs in a controlled environment. It effectively handled runtime errors, syntax issues, and edge cases using predefined test cases. The module demonstrated reliable performance with minimal execution delay, ensuring that users received quick validation of their code. This immediate feedback plays a crucial role in maintaining user engagement and iterative learning.

5.3 Accuracy of AI-Based Analysis :

The fine-tuned analytical model showed consistent performance in identifying time complexity, space complexity, and structural patterns in the code. It was able to differentiate between efficient and inefficient implementations of similar problems, providing structured outputs for evaluation. The accuracy of complexity estimation was observed to be significantly higher compared to traditional rule-based approaches, especially for non-trivial code patterns.

The generative AI component complemented this analysis by producing clear and context-aware explanations. It successfully translated technical insights into understandable feedback, helping users grasp the reasoning behind the evaluation. In most cases, the suggestions provided were relevant and aligned with standard programming practices, thereby supporting conceptual learning.

5.4 Impact of Hybrid AI Approach :

The integration of analytical and generative models proved to be a key strength of the system. While the fine-tuned model ensured precision in technical evaluation, the generative model enhanced interpretability and user engagement. This hybrid approach resulted in more comprehensive feedback compared to systems relying on a single method.

Users were able to not only identify errors and inefficiencies in their code but also understand how to improve them. The combined output reduced ambiguity and provided actionable insights, which is essential for educational platforms. This demonstrates that the hybrid model effectively bridges the gap between automated assessment and pedagogy.

5.5 Leaderboard and Engagement Analysis :

The inclusion of a performance-based leaderboard encouraged active participation and continuous improvement among users. By assigning points based on correctness, efficiency, and code quality, the system motivated users to optimize their solutions rather than focusing solely on passing test cases. This competitive element contributed to increased engagement and repeated usage of the platform.

5.6 Discussion :

The results indicate that the proposed system successfully enhances traditional code assessment methods by incorporating intelligent analysis and educational feedback. The hybrid AI approach addresses the limitations of standalone systems by combining accuracy with explainability. However, certain challenges remain, such as handling highly complex or unconventional coding styles and ensuring consistent performance across different programming languages.

Overall, the platform demonstrates strong potential as an effective educational tool for programming learning environments. It not only automates evaluation but also actively contributes to skill development, making it a valuable addition to modern educational technology.

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion :

This paper presented an educational code assessment platform that integrates a hybrid artificial intelligence approach to improve both code evaluation and learning outcomes. Unlike traditional systems that focus primarily on correctness, the proposed platform extends evaluation to include code efficiency, structure, and conceptual clarity. By combining a fine-tuned analytical model with a generative AI model, the system is able to deliver precise technical analysis along with human-readable explanations and suggestions. The results demonstrate that the hybrid approach enhances the overall quality of feedback, making it more comprehensive and useful for learners. The analytical component ensures accurate assessment of time and space complexity, while the generative component improves interpretability and supports conceptual understanding. Additionally, features such as automated execution, submission tracking, and leaderboard integration contribute to user engagement and continuous learning.

Overall, the proposed system successfully bridges the gap between automated code assessment and effective pedagogy. It provides a scalable and intelligent solution that can be utilized in academic institutions as well as self-learning environments to enhance programming education.

B. Future Scope :

Although the system demonstrates promising results, several opportunities exist for further improvement and expansion. Future

work can focus on extending support to multiple programming languages, enabling the platform to cater to a wider audience. Enhancing the fine-tuned model with larger and more diverse datasets can further improve the accuracy of complexity analysis and code evaluation.

Another potential direction is the incorporation of real-time feedback during code writing, allowing users to receive suggestions as they develop their programs. Integration of advanced features such as plagiarism detection, code similarity analysis, and adaptive difficulty levels can further strengthen the platform's capabilities.

Additionally, the system can be enhanced by incorporating personalized learning paths based on user performance and behavior. This would enable tailored recommendations and targeted practice for individual learners. Further research can also explore the use of more advanced hybrid AI architectures to improve efficiency and reduce computational overhead.

In conclusion, the proposed platform lays a strong foundation for intelligent and interactive programming education systems. With continued development and refinement, it has the potential to significantly transform the way coding skills are taught, evaluated, and improved.

VII. REFERENCES

- [1] Kumar and S. Singh, "Automated Code Evaluation Systems in Programming Education: A Review," *International Journal of Computer Applications*, vol. 182, no. 45, pp. 12–18, 2020.
- [2] Y. Liu, A. Jain, and K. Srinivasan, "Hybrid AI Approaches for Code Analysis and Optimization," *IEEE Access*, vol. 11, pp. 45000–45015, 2023.
- [3] P. Brown et al., "Language Models are Capable of Code Understanding and Generation," *Proceedings of the ACM Conference on AI and Software Engineering*, pp. 1–10, 2021.
- [4] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.
- [5] K. Patel and D. Shah, "Gamification in Education: Enhancing Student Engagement through Leaderboards and Reward Systems," *International Journal of Educational Technology*, vol. 8, no. 2, pp. 55–63, 2022.
- [6] S. Verma and A. Mehta, "Online Coding Platforms for Skill Development: A Comparative Study," *Journal of Educational Computing Research*, vol. 58, no. 4, pp. 789–805, 2020.